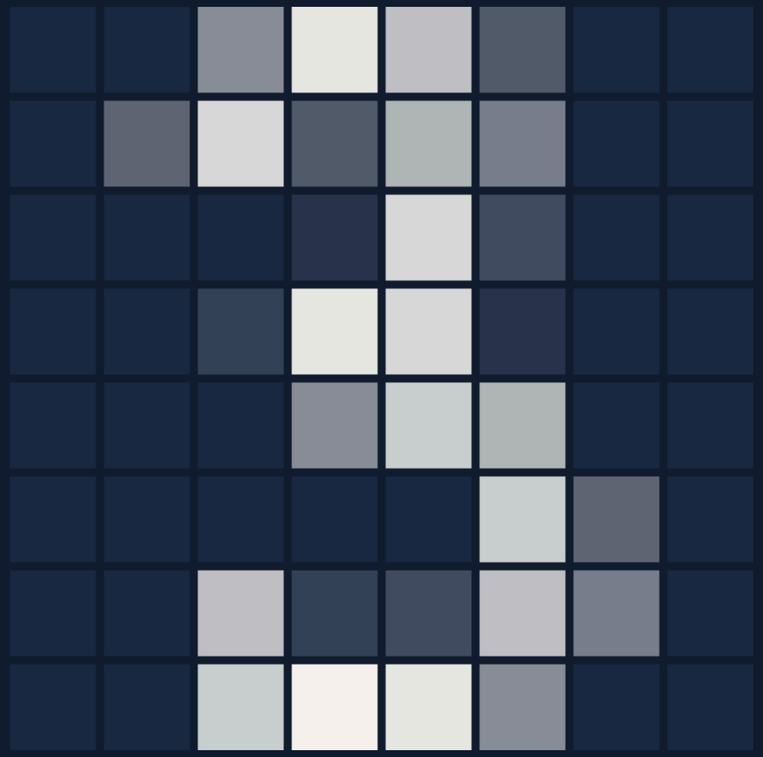
CS4052 · COMPUTACIÓN PARALELA Y DISTRIBUIDA · ENTREGA FINAL

KNN en paralelo con MPI

Tres versiones, mediciones con p y n variables, y cuánto ganamos al repartir el trabajo.



Johar Barzola · Leonardo Huaman · Luis Izaguirre

Grupo 7 · Caso 5 · Prof. José Fiestas · 2026-II

EN RESUMEN

Lo que hicimos para la entrega final

3 versiones

Mismas 360 predicciones que el código base con p = 1 a 8, también la mejora con reduce.

4 tamaños × 5 p

De n = 449 a n = 28 752 imágenes, con 1, 2, 4, 6 y 8 procesos.

$R^2 = 0{,}998$

El modelo teórico normalizado reproduce los tiempos medidos (error medio de 8 %).

p óptimo ≈ 18

Para n = 1797; crece más o menos con la raíz del tamaño de entrenamiento.

DE LA PARCIAL A LA FINAL

Lo que pidió el profesor y dónde está

PIDIÓ

Un pseudocódigo que no dependa de p, para ver fácil el span.

→

LO RESOLVIMOS ASÍ

PRAM con dos árboles explícitos: span O(log n).

diap. 6

PIDIÓ

Un análisis de Foster, además del DAG.

→

LO RESOLVIMOS ASÍ

Cuatro pasos con canales; la aglomeración lleva del PRAM al DAG.

diap. 5

PIDIÓ

La ecuación de eficiencia con un solo n.

→

LO RESOLVIMOS ASÍ

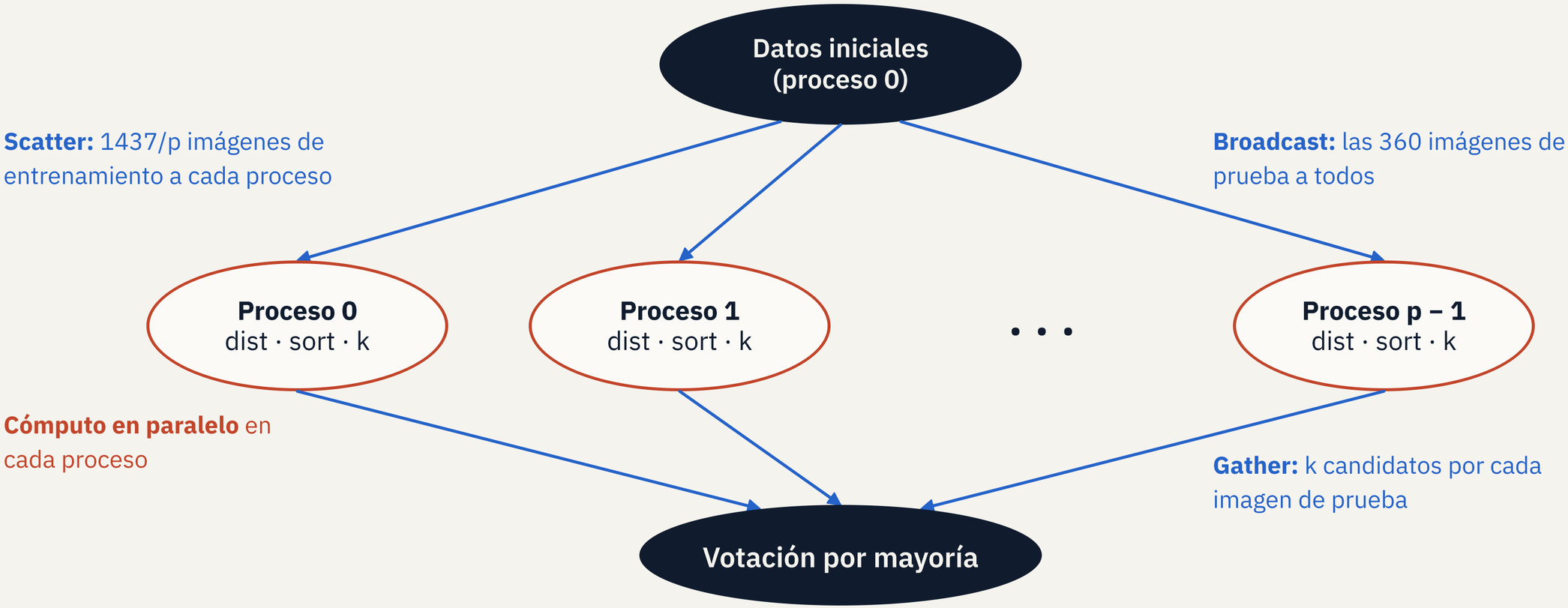
$n_{tr} = 0{,}8n$ y $n_{te} = 0{,}2n$:
$E = O(1/(1 + k \cdot p^2/n))$ y $n = \Theta(k \cdot p^2)$.

diap. 7 y 14

Además: medimos todo de nuevo en una sola sesión, usamos los costos O(log p) de MPI y agregamos la mejora con reduce.

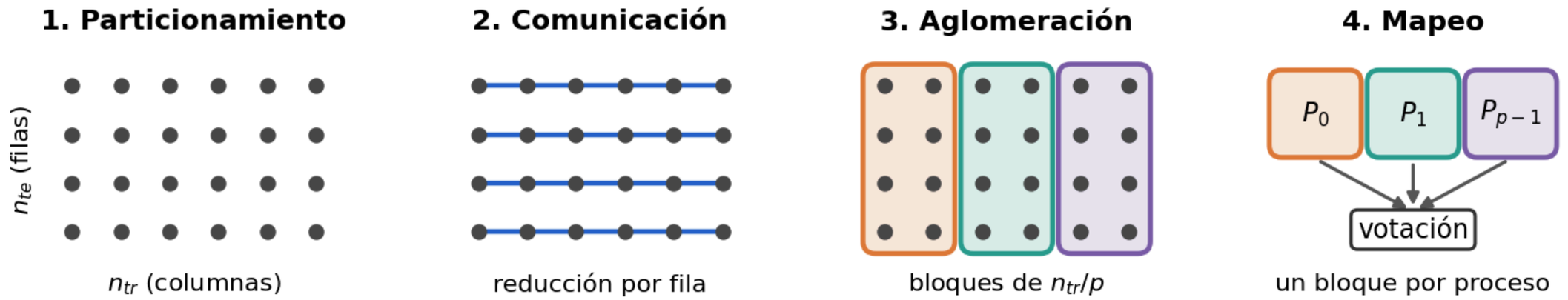
LA SOLUCIÓN · SEGÚN EL DAG DEL ENUNCIADO

Cómo se reparte el trabajo



DISEÑO · MÉTODO DE FOSTER

De medio millón de tareas a p bloques



01 · Particionamiento

Una tarea = una distancia: 517 320 tareas de costo O(d).

02 · Comunicación

Reducción por fila: del orden de n_tr·n_te canales. Es overhead.

03 · Aglomeración

p bloques que devuelven k candidatos: unos 3p canales.

04 · Mapeo

Un bloque por proceso; difieren en a lo más una fila.

Trabajo y span

```
1  for all (i, j) ∈ {1..n_te} × {1..n_tr} pardo
2    D[i,j] ← distancia(X_te[i], X_tr[j]);  C[i,j] ← {j}
3  for h = 1 to log₂ n_tr          // árbol 1
4    for all (i, j), j ≤ n_tr/2ʰ pardo
5      C[i,j] ← k menores de C[i,2j-1] u C[i,2j]
6  for all i = 1 to n_te pardo
7    ŷ[i] ← voto en C[i,1];  A[i] ← 1 si acierta, si no 0
8  for h = 1 to log₂ n_te          // árbol 2
9    for all i ≤ n_te/2ʰ pardo:  A[i] ← A[2i-1] + A[2i]
10 accuracy ← A[1] / n_te
```

Sin p: un procesador por tarea; con relleno, n_tr y n_te son potencias de 2.

$$W(n) = O(n^2) \qquad T_\infty(n) = O(\log n) \qquad T_p = O\left(\frac{n^2}{p} + \log n\right)$$

Trabajo W(n) = O(n²)

Todas las distancias y las mezclas: lo mismo que el mejor secuencial.

Span T∞(n) = O(log n)

Una distancia y los dos árboles: $\log_2$ n_tr + $\log_2$ n_te niveles.

Brent: T_p = O(n²/p + log n)

Casi todo es paralelizable; la comunicación entra recién en el DAG.

MÉTRICAS · UN SOLO N

Costo de cada paso y eficiencia

Paso del DAG	Costo, con un solo n
1 · Broadcast de la prueba	$O(\log p \cdot (a + n \cdot b))$
2 · Scatter del entrenamiento	$O(a \cdot \log p + n \cdot b)$
3–4 · Distancias y orden	$\approx O(n^2/p)$
5 · Gather de candidatos	$O(a \cdot \log p + n \cdot k \cdot p \cdot b)$
6–7 · Mezcla y voto (maestro)	$O(n \cdot k \cdot p)$

$$T_p(n, p) = O(n^2/p + n \cdot k \cdot p + a \cdot \log p + b \cdot n \cdot (1 + \log p + k \cdot p))$$

a = latencia y b = tiempo por dato, como en el DAG. El DAG del enunciado usa a·p (topología lineal).

Un solo n

n_tr = 0,8n y n_te = 0,2n: con el split 80/20 basta un n.

$E = O(1/(1 + k \cdot p^2/n))$

El overhead que manda es el del gather y la mezcla en el maestro.

Scatter y gather en árbol

Latencia a·log p, como en MPI (U2.4); E y la isoeficiencia no cambian.

DESARROLLO DEL CÓDIGO · INCISO B

Tres versiones beta

Beta 1 · 1034 ms

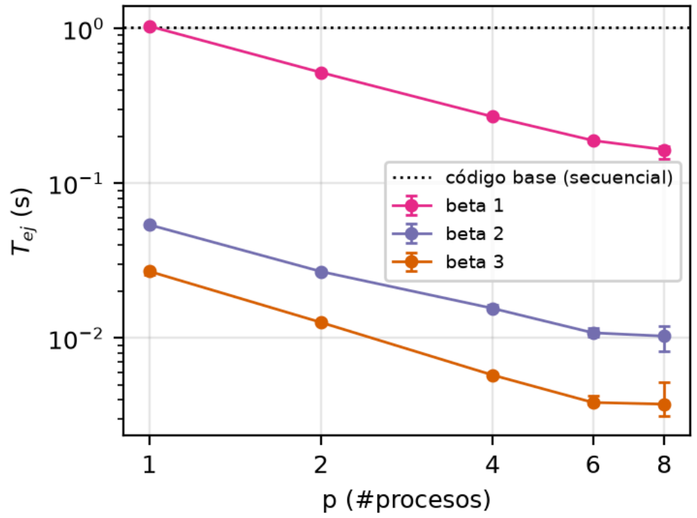
Traduce el algoritmo del DAG con el bucle de Python del código base.

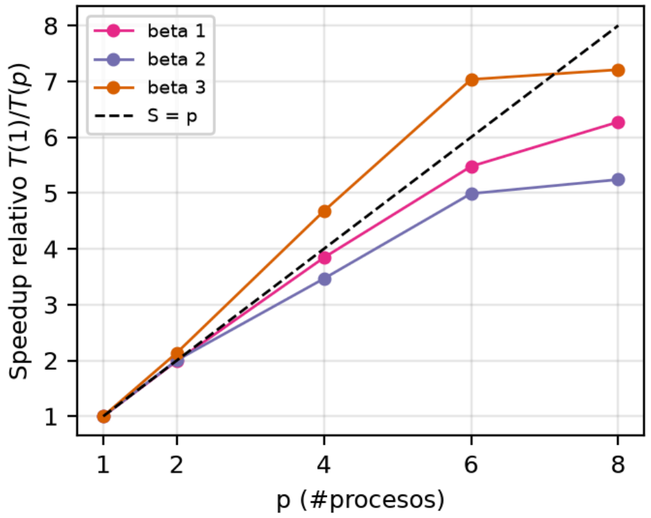
Beta 2 · 54 ms

Distancias con NumPy; con ella empezamos a medir por fases.

Beta 3 · 27 ms (final)

Producto de matrices por bloques y tamaño variable del problema.





Validación: 49 de 49 casos iguales al código base o al secuencial

DISEÑO EXPERIMENTAL

Cómo medimos

Máquina

Apple M1 Pro: 8 núcleos de rendimiento y 2 de eficiencia.

p = 1, 2, 4, 6 y 8: hasta un proceso por núcleo, con un hilo cada uno.

Tamaños

n/4 = 449 · n0 = 1797 · 4n = 7188 · 16n = 28 752.

Réplicas con ruido hechas después del split: sin mezclar entrenamiento y prueba.

Medición

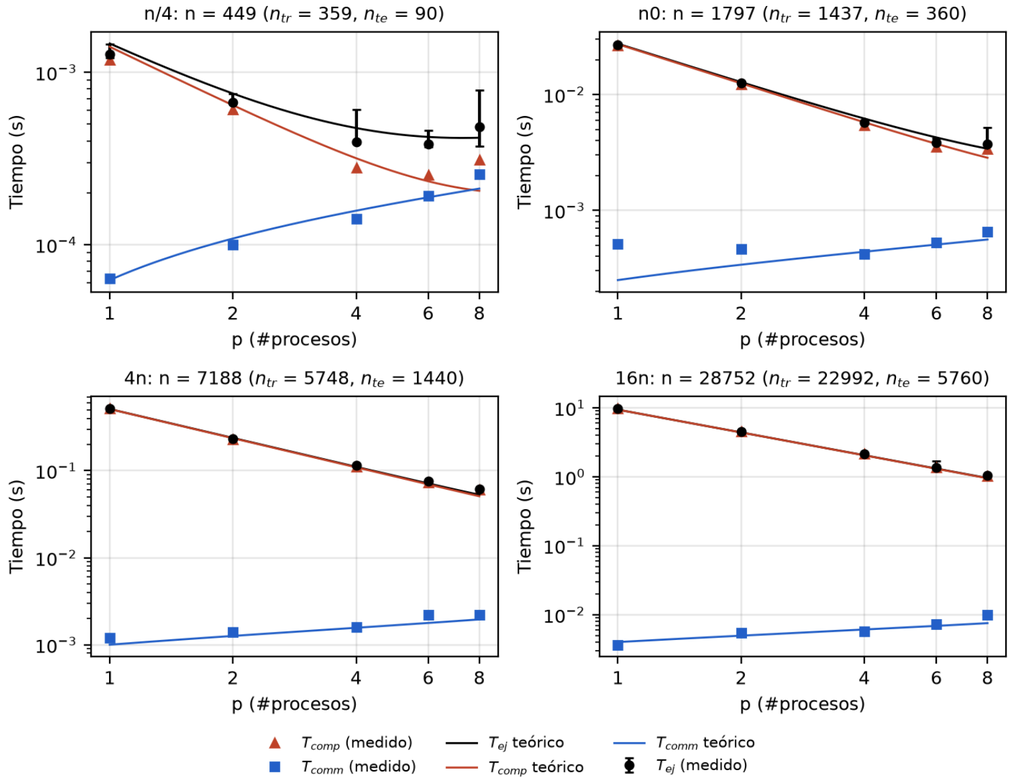
MPI.Wtime en el maestro; un mpirun por repetición, 4 intercaladas y la mediana.

Modo A sin barreras para T_ej; modo B con barreras para separar las fases.

T_s = la beta 3 en un solo proceso y sin comunicación: así S compara el mismo cálculo repartido entre p procesos.

RESULTADOS · INCISO C

Tiempos: ejecución, cómputo y comunicación



El cómputo baja como 1/p

Cada proceso trabaja con 1/p del entrenamiento.

La comunicación crece poco con p

En 4n y 16n es de 27 a 2700 veces menor que el cómputo: compute bound.

Con n/4 y p = 8 se empatan

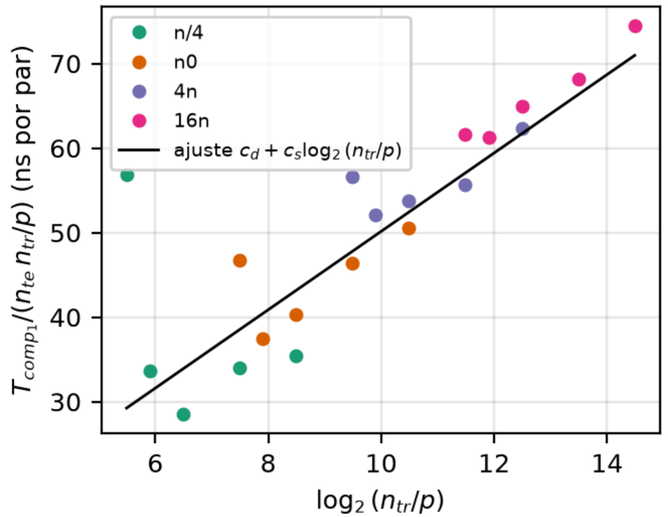
Comunicación ≈ cómputo, y el tiempo total vuelve a subir.

Marcadores: mediana medida · líneas: modelo normalizado

Teoría normalizada frente a lo medido

$$T_p^{teo} = n_{te}\,\frac{n_{tr}}{p}\left(c_d + c_s \log_2 \frac{n_{tr}}{p}\right) + c_2\, n_{te}\, k\, p \;+\; \log_2 p\,(a_1 + n_{te} b_1) + a_2 \log_2 p + n_{tr} b_2 + a_3 \log_2 p + p\, n_{te}\, k\, b_3$$

Cada término lleva una constante física, ajustada con su propia fase (modo B); scatter y gather con latencia log p.



R² = 0,998 · error medio 8 %

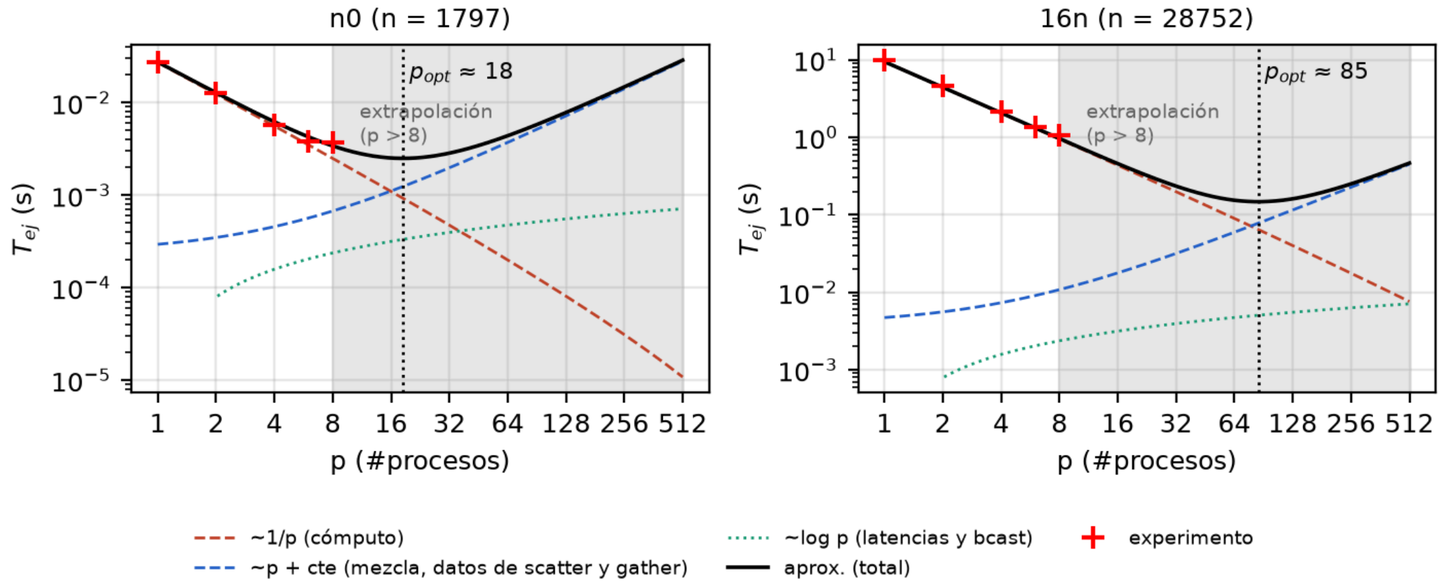
El ajuste usa solo las fases; un ping-pong confirma el costo por dato (≈ 2 ns).

Lo que no esperábamos: ordenar pesa más que calcular distancias

Ordenar cuesta ≈ 46 ns por par y la distancia ≈ 5 ns: el costo por par crece con $\log_2$(n_tr/p).

RESULTADOS · INCISO C

Cantidad óptima de procesos



$$T'(p) = -\frac{A}{p^2} + B + \frac{C}{p \ln 2} = 0 \;\Rightarrow\; p_{opt} \approx \sqrt{\frac{A}{B}} \;\propto\; \sqrt{n_{tr}}$$

n = 1797 (enunciado)

p óptimo ≈ 18

Aquí usamos 8, y de 6 a 8 solo ahorramos 0,1 ms.

n = 449: ≈ 7

Mínimo medido en p = 6.

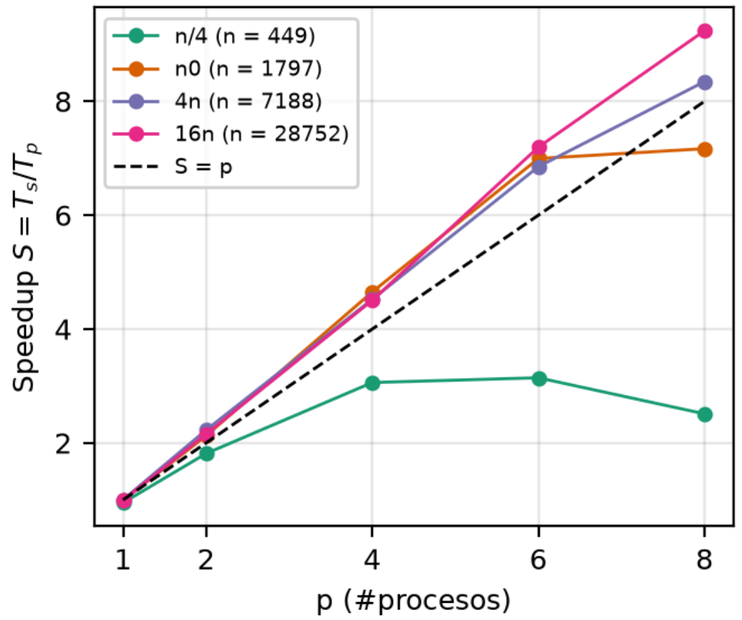
4n y 16n: ≈ 40 y 85

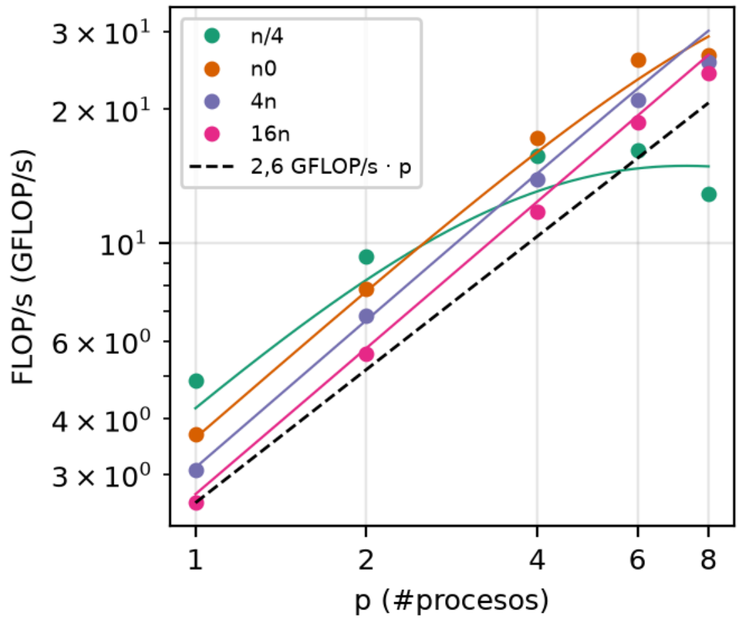
Más de 8 núcleos: conviene usarlos todos.

Zona gris: extrapolación del modelo (p > 8)

RESULTADOS · INCISO D

Speedup y FLOP/s





S = 9,2

con p = 8 en 16n: superlineal.

27 GFLOP/s

con p = 8; 3d = 192 FLOP por par.

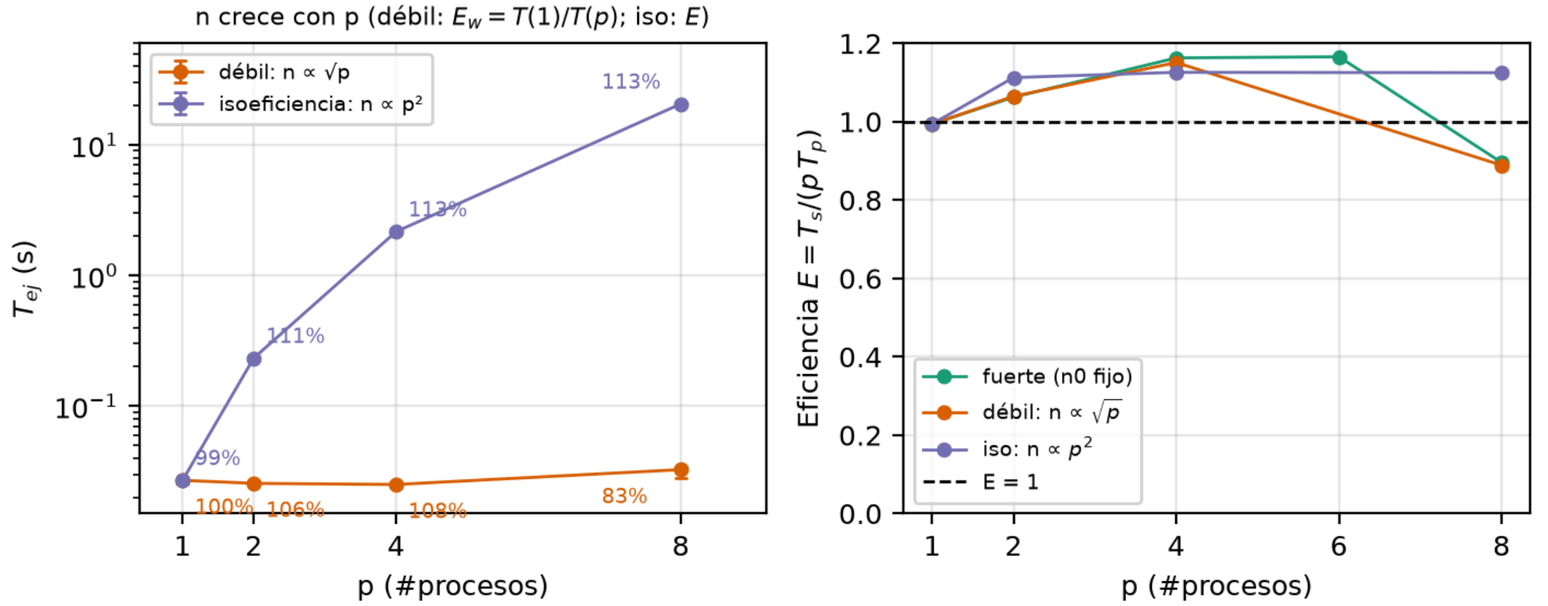
¿Por qué superlineal? Cada proceso ordena listas de n_tr/p (22 992/p en 16n) y T_s, de n_tr; el modelo lo predice.

Escalabilidad

E = O(1/(1 + k·p²/n))

un solo n: n_tr = 0,8n y n_te = 0,2n

$$W = K\,T_0 \;\Rightarrow\; n = \Theta(k\,p^2) \;\Rightarrow\; W = \Theta(k^2 p^4)$$



Fuerte: no escala

n0 fijo: E cae de 1,17 a 0,90 entre p = 6 y 8.

Débil: n ~ √p

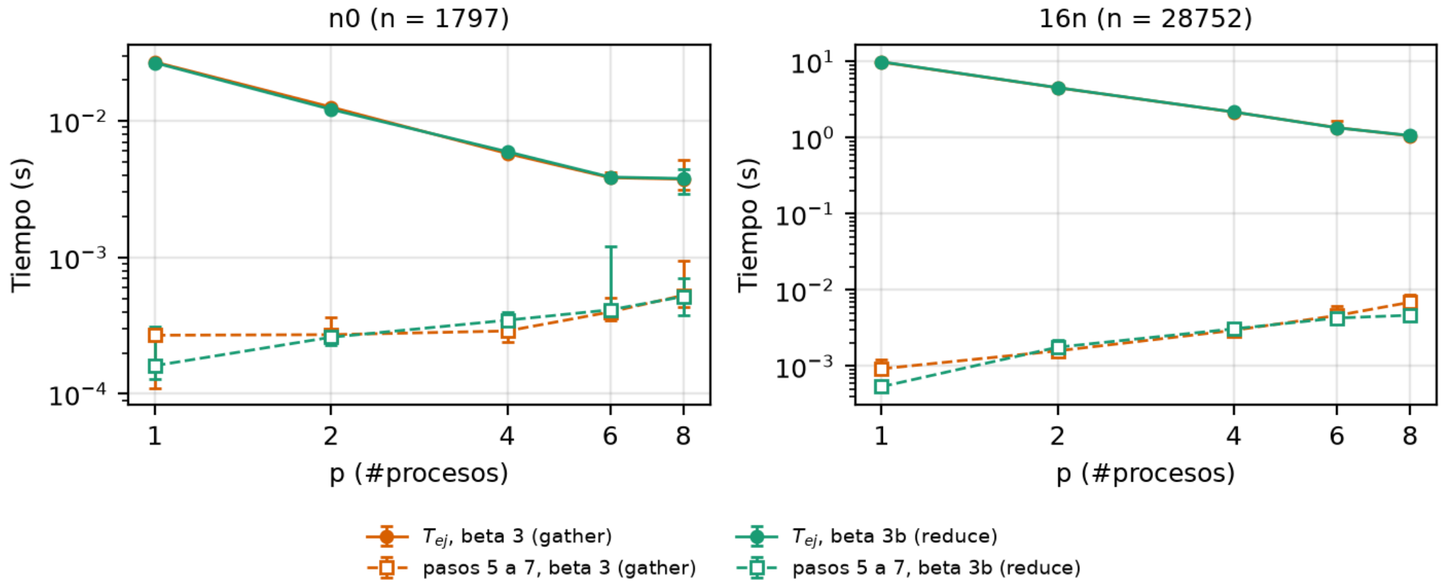
Trabajo por proceso fijo: E_w = 1,08 con p = 4 y 0,83 con p = 8.

Iso: n ~ k·p²

E plana: 0,99 · 1,11 · 1,13 · 1,13. Hasta 115 008 imágenes.

MEJORA · BETA 3B

Gather frente a reducción en árbol



Teoría: la mezcla pasa de n·k·p a n·k·log p, y la isoeficiencia de Θ(k·p²) a Θ(k·p·log p).

Con comm.reduce

−33 %

en los pasos 5 a 7 con 16n y p = 8 (6,9 → 4,6 ms).

Mismas predicciones

15 de 15 casos, igual que la beta 3.

Tiempo total: ±4 %

En n0, 4n y 16n (n/4: hasta +9 %). El ahorro es pequeño.

ANÁLISIS

Diferencias con la teoría y mejoras

Por qué no calza exacto

- Ordenar pesa más que calcular distancias.
- Con p = 8 el cómputo local sube 24 %: hardware compartido.
- pickle: latencia 10 veces mayor que con búfer.
- La mezcla del maestro crece con p.

Mejoras

- Seleccionar los k menores en vez de ordenar todo.
- Bcast, Scatterv y Gatherv con búfer, sin pickle.
- Que cada proceso lea su bloque (sin scatter).
- Más núcleos: para n = 1797 el óptimo es ≈ 18.

EN UNA FRASE

Mismas predicciones, compute bound, y escala solo si los datos crecen más rápido que los procesos.

Precisión 0,9833 con p = 1 a 8

p óptimo ≈ 18 para n = 1797

$n = \Theta(k \cdot p^2)$

BIBLIOGRAFÍA

Fuentes y uso de IA

Enunciado, código base y correo

El DAG, lo que pide el informe, la validación y los costos O(log p).

Diapositivas del curso, PD01 y PD02

PRAM, S, E, α + xβ, granularidad e híbrido.

U6.1 y U6.2 (2026-I) y knn.cpp

FLOP/s, T0, isoeficiencia, p óptimo y datos sintéticos.

mpi4py y scikit-learn

Colectivas, MPI.Wtime y los datos load_digits.

Uso de IA

Claude Opus 5 y 5.5

Escribió el código de las betas, la mejora y los scripts, ejecutó las mediciones y redactó los borradores del informe y de esta presentación.

Lo validamos con las predicciones del código base y revisamos todo con el material del curso.

Gracias

¿Preguntas?

Johar Barzola · Leonardo Huaman · Luis Izaguirre

Grupo 7 · CS4052 Computación Paralela y Distribuida · 2026-II

